

AFT Compiler Project

Final Report

Akshaya Tharun C R Aneesh Kadiyala Jakkala Naga Rohith Kumar
CS23BTECH11003 EE23BTECH11007 CS23BTECH11021

Rithvik Jami Sai Rishi Kasturi
CS23BTECH11022 CS23BTECH11024

Muvvala Sairam Suhas Panikar
CS23BTECH11038 CS23BTECH11040

November 28, 2025

Contents

1	Introduction	3
2	Language Tutorial	3
2.1	Hello World	3
2.2	Complex Math	3
2.3	Vector Processing	3
3	Language Reference Manual	4
3.1	Types	4
3.2	Scopes and Variables	4
4	Project Plan	4
5	Language Evolution	6
5.1	The Vector Problem	6
5.2	Complex Numbers	6
6	Compiler Architecture	6
6.1	Scanner and Parser	6
6.2	Abstract Syntax Tree (AST)	6
6.3	Semantic Analysis	7
6.4	Code Generation (Codegen)	7
7	Development Environment	7

7.1	Tools	7
7.2	Debugging	7
8	Test Plan and Test Suites	8
8.1	Unit Tests	8
8.2	Integration Tests	8
9	Results and Known Limitations	8
9.1	What Works	8
9.2	Limitations (Bugs)	8
A	Project Directory Structure	9
A.1	Frontend: Parsing (<code>parser/</code>)	9
A.2	Abstract Syntax Tree (<code>AST_gen/</code>)	9
A.3	Middle-end: Semantics (<code>SemanticAnalyser/</code>)	9
A.4	Backend: Code Generation (<code>CodeGen/</code>)	9
A.5	Runtime Environment	10
A.6	Testing Suite (<code>testcases/</code>)	10
A.7	Build System	10

1 Introduction

AFT is a programming language we designed for scientific computing and signal processing. We noticed that doing math in C++ is very verbose, and Python is slow for loops. So, we made AFT. It is a statically typed language that compiles to LLVM IR, which means it is fast.

The main reason we built this is to handle Complex Numbers ('c64') and Vectors naturally so that it could be use for signal processing by also adding functions for signal processing. In C++, you need libraries for this. In AFT, you just write '3+4j'.

This report explains how we built the whole system, from the Lexer 'aft.l' all the way to the Code Generator 'codegen.cpp' and the C/C++ runtime.

2 Language Tutorial

Here is how you write code in AFT. It looks a bit like Rust or Swift but simpler.

2.1 Hello World

```
1 fn main() i32 {  
2     print("Hello, AFT Compiler!");  
3     return 0;  
4 }
```

2.2 Complex Math

This is the best feature. We use 'j' for imaginary parts.

```
1 fn math_test() c64 {  
2     let a: c64 = 1.5 + 2.0j;  
3     let b: c64 = 0.5 - 1.0j;  
4  
5     // We can do standard math  
6     let sum: c64 = a + b;  
7  
8     let mul: c64 = a * b;  
9  
10    return sum;  
11 }
```

2.3 Vector Processing

We added a special operator '.*' for convolution because our runtime supports it.

```
1 fn signal_process() i32 {  
2     let signal: [i64] = [1, 2, 3];  
3     let kernel: [i64] = [0, 1, 0];  
4  
5     // Convolution happens in C runtime  
6     let result: [i64] = signal .* kernel;  
7  
8     // We can also reverse vectors
```

```

9     let rev: [i64] = ~signal;
10
11     return 0;
12 }

```

3 Language Reference Manual

3.1 Types

Our type system is very strict. We have many integer types because we use LLVM primitives.

- **Integers:** ‘i8’, ‘i16’, ‘i32’, ‘i64’, ‘i128’ (signed). Also unsigned versions ‘u8’...‘u128’.
- **Floats:** ‘f32’, ‘f64’.
- **Complex:** ‘c32’ (two floats), ‘c64’ (two doubles).
- **Vectors:** ‘[type]’, like ‘[i32]’ or ‘[c64]’.

3.2 Scopes and Variables

We use ‘let’ for variables and ‘const’ for constants.

- ‘let x: i32 = 10;’ (Mutable)
- ‘const PI: f64 = 3.14;’ (Immutable)

We support block scoping. If you define a variable inside an ‘if’ block, it disappears after the block ends.

4 Project Plan

The development of the AFT compiler followed a phase-based structure. Although we initially intended a strictly sequential workflow, time constraints and integration issues forced several phases to overlap. Below, we present the final project plan as it unfolded in practice.

Phase 1: Lexing and Parsing (Flex & Bison)

The first phase focused on building the lexical analyzer (`aft.1`) and grammar specification (`aft.y`). We defined tokens for all language constructs, including complex literals (COMPLEX) and custom operators such as convolution (`.*`). This phase established the syntactic foundation of the language, but due to time constraints, we could not test every grammar rule extensively before moving to the next stage.

Phase 2: Abstract Syntax Tree (AST) Construction

In the second phase, we designed the AST hierarchy (`AST.h`) and implemented node classes for expressions, statements, functions, and vector constructs. To validate correctness, we built a dedicated visualization tool, `print_ast.cpp`, which prints the gen-

erated tree structure. This helped us verify that the parser reduced grammar rules properly. However, incomplete test coverage from Phase 1 occasionally caused incorrect AST shapes, which we discovered only during semantic or code-generation work later.

Phase 3: Semantic Analysis

Semantic analysis (`semanticAnalyzer.cpp`) was developed in parallel with Phase 4 due to schedule pressure. The symbol table, type inference logic, and scope management were implemented during this stage.

A major challenge was that semantic analysis depended heavily on the correctness of earlier phases. Because some grammar rules and AST nodes had not been fully tested beforehand, semantically valid programs were sometimes rejected due to issues originating in parsing or AST construction. Debugging these problems required repeatedly revisiting earlier phases, modifying node definitions, adding missing fields, or adjusting AST structure—significantly increasing integration effort.

Furthermore, integrating built-in functions into the symbol table was nontrivial. We needed the analyzers to recognize builtins so codegen could emit calls to runtime implementations without raising “undeclared function” errors.

Phase 4: Code Generation (LLVM IR) and Runtime Integration

Phase 4 involved lowering the AST into LLVM IR (`codegen.cpp` and `builtin_codegen.cpp`). We initially attempted to implement our built-in functions (e.g., `sin`, `cos`, vector operations) directly in AFT’s own `.af` code. However, this approach was both inefficient and difficult to maintain, particularly for complex numbers and vector manipulation.

Therefore, we shifted to a hybrid runtime approach:

- `vector_runtime.c` handles vector allocation, resizing, and convolution.
- `builtins.cpp` wraps standard mathematical operations using C/C++ and exposes them through `extern "C"` interfaces.

The code generator then emits LLVM IR that calls these runtime functions. This greatly improved performance and reliability, but required coordination between multiple components.

Integration Challenges and Limitations

Several difficulties emerged during integration:

- **Cross-phase dependencies:** Semantic analysis required parser correctness, and codegen required semantic stability. Because these phases were developed in parallel, inconsistencies often surfaced late.
- **Coordination issues:** Different team members worked on different subsystems at different points in time, making it difficult to recall design decisions or track structural changes across AST, parsing, or semantic logic.

- **Frequent structural revisions:** Whenever a missing case was discovered, we often had to modify AST node definitions or add new fields, which cascaded into updates in semantic analysis and code generation.
- **Symbol-table handling for builtins:** Ensuring that builtins were semantically recognized while still lowering correctly to C/C++ runtime code was particularly challenging.
- **Limited testing:** Due to deadlines, extensive unit and integration testing for each phase was not possible. Many issues only appeared when later phases exercised earlier components.

Despite these challenges, we were able to build a functioning compiler pipeline capable of parsing, analyzing, generating LLVM IR, and linking with a custom runtime to execute AFT programs.

5 Language Evolution

At first, AFT was just a calculator. But then we realized we needed arrays.

5.1 The Vector Problem

Originally, we just wanted fixed arrays. But for signal processing, arrays change size. So we changed ‘VectorType’ in the AST to support dynamic sizes. This forced us to write ‘vec_create’ and ‘vec_push’ in C and link it to our compiler.

5.2 Complex Numbers

We debated if Complex numbers should be a struct or a primitive. We decided to make them primitive (‘E_C64’ in ‘elem_ids.h’). This makes the parser easier, but the Codegen harder because we have to manually do the math (real part + imaginary part) in ‘builtin_codegen.cpp’.

6 Compiler Architecture

Our architecture is a pipeline: Code → Flex → Bison → AST → Semantics → LLVM IR.

6.1 Scanner and Parser

The scanner (‘aft.l’) detects things like ‘3j’ as a ‘COMPLEX’ token. The parser (‘aft.y’) builds the AST. We handle precedence carefully. For example, ‘**’ (exponentiate) has higher precedence than ‘*’ (multiply).

6.2 Abstract Syntax Tree (AST)

Our AST is a tree of C++ classes. The base class is ‘ASTNode’.

- ‘StatementBlock’: Holds a list of statements.

- ‘BinaryExpression’: Holds ‘left’ and ‘right’ expressions.
- ‘VectorLiteralExpr’: This was tricky. It holds a list of expressions. In ‘codegen’, we have to calculate the memory size for this.

6.3 Semantic Analysis

The ‘SemanticAnalyzer’ is the police of our compiler.

- It has a ‘SymbolTable’ stack. When we enter a function, we push a scope. When we leave, we pop it.
- It performs **Type Inference**. If you write ‘let x = 10;’, it guesses ‘x’ is ‘i64’.
- It checks validity. For example, you cannot do ‘convolution’ on scalars. The ‘check_binary_operation’ function ensures both sides are vectors.

6.4 Code Generation (Codegen)

The ‘codegen.cpp’ file does the heavy lifting.

1. **Entry Block:** We use ‘createEntryAlloca’ to put all variables at the top of the function. This is an LLVM requirement for ‘mem2reg’ optimization.
2. **Complex Numbers:** LLVM has no complex type. In ‘builtin_codegen.cpp’, we treat complex numbers as ‘double, double’ structs. We implemented ‘promoteToComplex’ to convert real numbers to complex if needed.
3. **Vectors:** Vectors are implemented as a pointer to a ‘VecHeader’ struct (defined in ‘vector_runtime.c’). The Codegen generates calls to ‘vec_create’, ‘vec_resize’, etc.

7 Development Environment

7.1 Tools

We used standard Linux tools:

- **Flex Bison:** For parsing.
- **LLVM 14+:** For code generation.
- **GCC/Clang:** To compile the runtime ‘vector_runtime.c’.

7.2 Debugging

We implemented a debugging utility named `print_ast.cpp` to visualize the internal structure of the Abstract Syntax Tree. This tool generates a readable, hierarchical representation of the nodes produced by the parser, which helped us confirm that the grammar rules, operator precedence, and associativity were being applied correctly during AST construction.

8 Test Plan and Test Suites

We tested the compiler by writing small ‘.af’ programs.

8.1 Unit Tests

- **Math:** → checked printing and other builtin functions as well like trigonometric functions and also other numeric functions like abs and pow.
- **Complex:** → Checked
- **Vectors:** ‘let v = [1, 2];’ → We checked if vector operations like reverse worked or not
- and many more and some nontrivial cases as well which you can see in the testcases directory
- written in detail in appendix when talking about testcases directory

In detail of what functions were implemented in whitepaper

8.2 Integration Tests

We linked the runtime ‘vector_runtime.o’ with our generated object file.

9 Results and Known Limitations

9.1 What Works

The compiler works for correct code. You can declare functions, do loops, use complex numbers, and print results. The ‘builtins.cpp’ file successfully connects C++ math functions (like ‘sin’, ‘cos’, ‘abs’, ‘pow’ etc which can be seen in the src file for builtins.cpp) to our language.

9.2 Limitations (Bugs)

We have some issues because the Semantic Analyzer is too strict.

- **Strict Typing:** (Intentional(Architectural design) but midway felt could be changed as testing itself was made difficult due to it) If you write ‘let v = [1, 2, 3]’, the compiler thinks it is ‘[i64]’. If you try to pass this to a function expecting ‘[f64]’, it gives an error. The type inference doesn’t automatically cast vector elements. You have to write ‘[1.0, 2.0, 3.0]’.
- if we have an empty vector it defaults to i64 and not to the element it gets concatenated to when it gets concatenated i.e semantic check fails here.
- **Memory:** We use ‘malloc’ for vectors but we don’t always ‘free’ them effectively. This is a future improvement.

A Project Directory Structure

The AFT compiler is organized into a modular directory structure, separating the frontend (parsing), middle-end (semantics), backend (codegen), and runtime libraries.

A.1 Frontend: Parsing (parser/)

This directory contains the lexical and syntactic analysis modules.

- `aft.l`: The Flex specification file. It defines the regex rules for tokenizing keywords (e.g., `let`, `c64`), literals, and operators.
- `aft.y`: The Bison grammar file. It defines the language grammar rules and constructs the initial AST nodes upon reduction.
- `aft.tab.c/h`: The C files generated by Bison, serving as the interface between the lexer and the AST.

A.2 Abstract Syntax Tree (AST_gen/)

This directory defines the intermediate representation of the code.

- `AST.h`: The core header file defining the polymorphic C++ class hierarchy (e.g., `Program`, `Statement`, `BinaryExpression`).
- `ast.cpp`: Implements the `print()` methods for the AST nodes, enabling tree visualization.
- `print_ast.cpp`: A standalone utility used during development to verify that the parser correctly builds the tree structure before code generation.

A.3 Middle-end: Semantics (SemanticAnalyser/)

This module ensures type safety and logical consistency.

- `semanticAnalyzer.h/cpp`: Implements the `SemanticAnalyzer` class. It manages the Symbol Table, enforces scope rules, and validates type compatibility (e.g., ensuring vectors are not mixed with incompatible scalars).

A.4 Backend: Code Generation (CodeGen/)

This module translates the AST into LLVM IR.

- `codegen.h/cpp`: The core transpiler. It traverses the AST and uses the LLVM C++ API to generate instructions. It handles control flow (loops, if-else) and variable allocation. It also takes care of the `vector_runtime.c` functions.
- `builtin_codegen.cpp`: Handles the generation of complex number and normal arithmetic, trigonometric functions (manually implementing operations like $(a + bi)(c + di)$) and calls to intrinsic math functions.

A.5 Runtime Environment

The AFT language relies on a hybrid C/C++ runtime to handle high-level features.

- `external_functions/`: Contains `vector_runtime.c`, which manages dynamic memory allocation (`malloc`, `realloc`) for the AFT vector type. It also defines `elem_ids.h` for runtime type identification.
- `run_time/`: Contains `builtins.cpp`, which wraps standard C++ math library functions (like `sin`, `cos`, `pow`) into `extern "C"` interfaces callable by the generated LLVM IR.

A.6 Testing Suite (`testcases/`)

A comprehensive suite of tests covering various language features.

- `till_ast/`: Tests to verify the parser and AST construction, It doesnt work properly for semantic or codegen(limtatations).
- `semantic_checks/`: Tests designed to fail, ensuring the semantic analyzer correctly catches type mismatches and scoping errors.
- `vec_operations/`: Integration tests specifically for vector creation, reversal (‘ ‘), vector reassigning etc.
- `loop_checks/`: Verifies the logic of `for` and `while` loops.
- `overall`: Non trivial testcases showing that out code works i.e compiles, checks semantically and outputs the respective .ll after codegen and also links the .o we get from .ll using `llc(clang)` with our runtime and builtin functions which are called from out .af code to these functions using intermediate codegen.

A.7 Build System

- `Makefile`: Orchestrates the compilation of the parser, AST, codegen, and runtime object files into the final compiler binary(main).
- `aft.sh`: A shell script wrapper that compiles a user’s ‘.af’ source file, links it with the runtime libraries, and produces an executable only after respective syntax and semantic checks work properly